Automated QA for the Eclypse platform

A system that uses the platform the way a client's team does, proves each action really worked, and tells the right person in plain language when something breaks, without touching client data or raising false alarms. The prototype runs it against a stand-in platform with one command: `npm run qa`.

1. The parts

- **Probe** is the real-user test run. A Playwright browser signs in with a one-time code, clicks, fills in forms, uploads and downloads files, follows a long AI job to the end, and checks every step against a known right answer. Its verdict never depends on an AI model.
- **Explore** is a read-only crawl that maps the app and proposes new checks for a person to approve. With an Anthropic key it uses Claude Haiku to decide what to open next; without one it uses built-in rules.
- **QA console** is where Eclypse engineers work. They add a workspace (a target URL), start Explore or Probe, watch the log live, read results, and connect a GitHub repository.
- **Stand-in** is a small copy of an AI platform for pharma teams, used only by the prototype. It holds two client companies, a synthetic QA company and a canary company, and has switches that inject real bugs.

2. When it runs

Trigger	Target	What runs	On failure
Every code change	Fresh build of the branch	Full Probe, AI mocked (free, true facts, varied wording)	Blocks the merge
After every release	Staging	Full Probe, real AI replayed from recordings	Halts promotion, owner told
Every 30 minutes	Live, synthetic company only	Read-only safe set of checks	Page only if failed twice in a row
Every 6 hours	Live, synthetic company only	Safe set plus one small real AI job	As above

Code changes reach it two ways: CI workflows on the repository, or the console watching a branch with an engineer's GitHub token and marking each commit pass or fail. The second works even where the team is only a collaborator on the repository and cannot install apps or webhooks.

3. What it checks

Outcomes, not page loads. The uploaded file appears with the right row count and survives a reload. Delete removes that document and nothing else. A saved setting is still there after a reload. An AI job reaches Complete and every number it reports matches the input file. Downloads match what was on screen. Wrong password, wrong code, wrong file type and a missing column each fail with a clear message.

Every test also runs inside a guard that blocks unexpected writes, scans every response and the final screen for another company's data, turns crashes and server errors into failures, and cleans up what the test created, even when it fails.

4. Tools, and why

Choice	Why	Not chosen
Playwright (TypeScript)	Real browsers, auto-waiting (the main defence against flaky tests), first-class uploads and downloads, network interception for the safety guard, traces for debugging. Same language as the frontend, so its engineers can maintain the tests.	Cypress: weaker on downloads and multi-origin. Selenium: more plumbing and flake.
Locators by role and label	Tests find controls the way users and screen readers do, so a restyle does not break them, and a missing label is itself a finding.	CSS and XPath selectors break on every redesign.
TOTP for the QA account	The robot computes the same code a phone app shows. No backdoor in the product.	Email codes: slow, needs inbox polling, flaky.
Claude Haiku, bounded	Drives Explore and suggests checks; optional plain-English failure notes. Cheap and fast. Never decides pass or fail.	AI in the pass/fail path: not repeatable, costs every run.

5. Signing in when sign-in needs a one-time code

- The QA account is a real user with real two-step sign-in. Its authenticator secret lives in the secret store for each environment, and the runner computes the six-digit code from it. Nothing in the product skips the code for tests.
- A code works once. The runner remembers the last time window it used, waits for a fresh one, and avoids the last two seconds of a window, so a slow page cannot fail on an expiring code.
- It signs in once per run and reuses the session. Fewer sign-ins means fewer codes and no lockouts. Runs on the same account never overlap.
- Secrets stay out of artifacts: tracing is off while secrets are typed, every report is redacted, and the session is signed out at the end so a captured token is useless.

6. Staying safe on the live platform

The rule: the robot can only see and change synthetic data, and it stops if it cannot prove that.

1. **Least privilege.** The QA account belongs to one synthetic company only, as a normal user. Client accounts are seeded with random credentials that are thrown away, so the robot cannot sign in as a client.
2. **Fail closed.** Right after sign-in it checks which company it landed in and stops the whole run if it is not the synthetic one.
3. **Write allow-list.** Every write leaving the browser is checked. Only expected writes pass, and a delete is only allowed for something this test created.
4. **Canaries, not client data.** A second synthetic company holds records with a unique marker. Seeing that marker anywhere, in any test, is a critical failure. Fetching its document by exact id must return 404.
5. **Interlocks.** Live runs refuse to start unless live mode is switched on, the host is on the allow-list and only live-safe checks are selected. Explore is read-only everywhere.
6. **Evidence.** Production keeps only the text report. No page content or screenshots go to an AI model.

7. AI output that changes on every run, and cost

Check facts, ignore wording. The robot uploads a synthetic file whose correct answers were worked out by hand. Every structured figure the AI reports must equal those answers, the summary must exist and contain no junk, and downloads must match the screen. The prose itself is never gated: unsupported numbers and an optional AI review only produce warnings.

Long jobs. The robot watches what the user sees and fails early and specifically, for example "stuck at Analyzing with AI, 75%, no change for 15 seconds", rather than after a vague ten-minute timeout.

Where	AI mode	Cost
Every code change	Mocked	Nothing
Release runs	Real model, recorded once, replayed by hash of model, prompt version and input	Close to nothing per rerun
Live, a few times a day	Real Claude Haiku on the smallest file	Well under a dollar a month
Explore	Haiku with a step cap, a token cap and a cache	Cents per run, shown on the job page

Retries are capped at one, since a retried AI job also costs money. A configured key that fails is a failure, never a silent fallback to the mock, which would make a broken integration look healthy.

8. What happens when it finds a problem

- **One problem, one entry.** Failures are grouped by cause. If sign-in breaks, the report says that 20 checks did not run, not 21 red lines.
- **Plain language.** Each entry gives the journey, the step, expected against actual in user terms, and the evidence: screenshot, trace and job timeline.
- **Routing.** Critical (data leak, safety stop): security and on-call at once. High (a journey is broken): the merge or release is blocked. Medium: a ticket. On a watched branch the commit is marked pass or fail with a link to the report.
- **No false alarms.** An unreachable environment is one result, not many failures. Live checks must fail twice before paging. A pass on retry is reported as flaky, not failed. Browser console noise is evidence, not a failure. A slow but working page must pass: the bug matrix proves it.

9. Risks the brief did not mention

- Framework noise: Next.js adds a hidden alert region that made "no error shown" checks fail on success. Checks are scoped to the page's main region.
- The QA account locked out by retried codes, blinding live monitoring. One sign-in per run, no overlapping runs, one account per environment.
- Bot traffic in analytics, billing and fraud rules. Bot requests are tagged and the account is known.
- An AI provider outage mistaken for a product bug. It is reported as an environment problem.
- Stale recordings after a prompt or model change. Both are part of the recording key.
- Canary drift: if the canary is deleted, isolation tests pass without proving anything. The fix is a daily check that it still exists, not yet in the prototype.
- Prompt injection during Explore: page text is treated as data, and the read-only guard blocks every write whatever the model chooses.

10. Upkeep, and what comes next

About half a day per sprint once stable, mostly when a flow changes. The QA team owns the framework; feature teams update the journey test in the same pull request that changes the journey.

Next: Probe written by AI and replayed without it (Haiku walks a journey once, the steps are recorded as a script, later runs replay the script, and a failing replay asks a person to approve a fix); credentials asked for only when a run reaches a sign-in screen; one synthetic shadow company per client for client-specific settings; cross-browser runs, an accessibility scan and visual checks on key screens.

11. Prototype results

Command	Result
`npm run qa`	PASS: 22 checks (sign-in plus 21 user journeys), about 60 seconds including a 20 second AI job.
`npm run qa:bugs`	10 of 10 injected bugs caught, each with a specific headline. The slow page passes: no false alarm.
`npm run qa:repeat`	10 runs with nothing changed, all identical, though the mock AI words its summary differently each time.
`npm run demo`	Starts the QA console with a ready workspace, so Explore and Probe can be tried from the browser.

AI tools: the code, tests and this document were written with Claude Code as a pair programmer. Every design decision and result was reviewed and run locally.